

UNIVERSITY OF ASIA PACIFIC
DEPARTMENT OF CSE

Lab Report - 3

COURSE CODE: CSE 402.

COURSE TITLE: OPERATING SYSTEM LAB.

SUBMITTED BY:

NAME : SYED MAHATASIN HUSSAIN.

STUDENT ID : 23101140

SEMESTER : 4th YEAR 1st SEMESTER

SECTION : C - 2

DATE : 16 - 08 - 2026

SUBMITTED TO

AITA RAHMAN ORTHI

LECTURER AT UAP

1. Problem:

Implement the following **CPU Scheduling Algorithms** for the given set of processes:

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	7
P2	1	4
P3	2	15
P4	3	11
P5	4	20
P6	4	9

Implement the following scheduling algorithms:

- **Round Robin (RR)** with **Time Quantum = 5**
- **Shortest Job First (SJF) – Non-Preemptive**
- **First Come First Serve (FCFS)**

For each scheduling algorithm, calculate:

- **Completion Time (CT)**
- **Turnaround Time (TAT)**
- **Waiting Time (WT)**
- **Average Turnaround Time (AVG TAT)**
- **Average Waiting Time (AVG WT)**

Also, represent the **process execution sequence/Gantt Chart** for each algorithm.

Finally, **compare the Average TAT and Average WT** of Round Robin, SJF, and FCFS and determine which scheduling algorithm performs better for the given set of processes. Explain the reason for the difference in performance.

2. Source Code:

```
# Round Robin (RR)
process = ["p1", "p2", "p3", "p4", "p5", "p6"]
at = [0, 1, 2, 3, 4, 4]
bt = [7, 4 , 15, 11, 20, 9]


n = len(process)
time_q = 5
rt = bt.copy()

ct = [0] * n
tat = [0] * n
wt = [0] * n
current_time = 0

queue = []
visited = [False] * n

for i in range(n):
    if at[i] <= current_time and visited[i] == False:
        queue.append(i)
        visited[i] = True

while len(queue) > 0:
    i = queue.pop(0)

    if rt[i] > time_q:
        current_time += time_q
        rt[i] -= time_q


    else:
        current_time += rt[i]
        rt[i] = 0
```

```
        ct[i] = current_time

    for j in range(n):
        if at[j] <= current_time and visited[j] == False:
            queue.append(j)
            visited[j] = True


    if rt[i] > 0:
        queue.append(i)

for i in range(n):
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]

print("Process\tAT\tBT\tCT\tTAT\tWT")

for i in range(n):
    print(process[i], "\t", at[i], "\t", bt[i], "\t", ct[i],
"\t", tat[i], "\t", wt[i])


rr_avg_tat = sum(tat)/n
rr_avg_wt = sum(wt)/n

print("\nAverage TAT =", rr_avg_tat)
print("Average WT =", rr_avg_wt)
print()

# Shortest Job First
process = ["p1", "p2", "p3", "p4", "p5", "p6"]
at = [0, 1, 2, 3, 4, 4]
bt = [7, 4 , 15, 11, 20, 9]

n = len(process)
```

```
ct = [0] * n
wt = [0] * n
done = [0] * n
tat = [0] * n

time = 0
completed = 0

while completed < n:

    x = -1
    for i in range(n):

        if at[i] <= time and done[i] == 0:

            if x == -1 or bt[i] < bt[x]:
                x = i

    if x == -1:
        time += 1

    else:
        time = time + bt[x]

        ct[x] = time
        done[x] = 1
        completed += 1


for i in range(n):
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]


print("Process\tAT\tBT\tCT\tTAT\tWT")
```

```
for i in range(n):
    print(process[i], "\t", at[i], "\t", bt[i], "\t", ct[i],
"\t", tat[i], "\t", wt[i])

sjf_avg_tat = sum(tat)/n
sjf_avg_wt = sum(wt)/n

print("\nAverage TAT =", sjf_avg_tat)
print("Average WT =", sjf_avg_wt)
print()

# First Come First Serve(FCFS)
lst = [
    ["P1",0,7],
    ["p2",1,4],
    ["p3",2,15],
    ["p4",3,11],
    ["p5",4,20],
    ["p6",4,9]
]

lst.sort(key=lambda x: x[1])


ct = []
tat = []
wt = []

time = 0

for i in lst:
    name, at, bt = i

    if time < at + 1 :
        time = at
```

```
    time += bt
    ct.append(time)

    turnaround = time - at
    tat.append(turnaround)

    waiting = turnaround - bt
    wt.append(waiting)

print("P_id\t AT\t BT \t CT \t TAT \t WT")

for i in range(len(lst)):
    print(lst[i][0], "\t",
          lst[i][1], "\t",
          lst[i][2], "\t",
          ct[i], "\t",
          tat[i], "\t",
          wt[i], "\t")

fcfs_avg_tat = sum(tat)/n
fcfs_avg_wt = sum(wt)/n
print("Average Tranaround time: ",fcfs_avg_tat)
print("Average Waiting time: ", fcfs_avg_wt)

print()


if rr_avg_tat < sjf_avg_tat and rr_avg_tat < fcfs_avg_tat:
    print("Round Robin is better TAT.")
elif sjf_avg_tat < rr_avg_tat and sjf_avg_tat < fcfs_avg_tat:
    print("Shortest Job First is better TAT.")
else:
    print("FCFS is better TAT.")
```

```python
if rr_avg_wt < sjf_avg_wt and rr_avg_wt < fcfs_avg_wt:
    print("Round Robin is better WT.")
elif sjf_avg_wt < rr_avg_wt and sjf_avg_wt < fcfs_avg_wt:
    print("Shortest Job First is better WT.")
else:
    print("FCFS is better WT.")
```

3. Output:

```
PROBLEMS 12   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SPELL CHECKER 12

PS C:\Users\hp\Desktop\CSE 402 Lab\Lab 4> & "C:\Program Files\Python313\python.
Process AT      BT      CT      TAT     WT
p1      0       7       31      31      24
p2      1       4       9       8       4
p3      2       15      55      53      38
p4      3       11      56      53      42
p5      4       20      66      62      42
p6      4       9       50      46      37

Average TAT = 42.166666666666664
Average WT = 31.166666666666668

Process AT      BT      CT      TAT     WT
p1      0       7       7       7       0
p2      1       4       11      10      6
p3      2       15      46      44      29
p4      3       11      31      28      17
p5      4       20      66      62      42
p6      4       9       20      16      7

Average TAT = 27.833333333333332
Average WT = 16.833333333333332

P_id    AT      BT      CT      TAT     WT
P1      0       7       7       7       0
p2      1       4       11      10      6
p3      2       15      26      24      9
p4      3       11      37      34      23
p5      4       20      57      53      33
p6      4       9       66      62      53
Average Tranaround time:  31.666666666666668
Average Waiting time:  20.666666666666668

Shortest Job First is better TAT.
Shortest Job First is better WT.
PS C:\Users\hp\Desktop\CSE 402 Lab\Lab 4>
```

4. Discussion:

In this experiment, **Round Robin (RR), Shortest Job First (SJF), and First Come First Serve (FCFS)** scheduling algorithms were implemented and compared using the same six processes. The comparison was based on **Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT)**.

For **Round Robin**, with a time quantum of **5 units**, the Average TAT is **42.17** and the Average WT is **31.17**. For **SJF**, the Average TAT is **27.83** and the Average WT is **16.83**, which are the lowest among the three algorithms. FCFS gives an Average TAT of **31.67** and an Average WT of **20.67**. Therefore, SJF performs better for this particular set of processes because it executes shorter jobs first and reduces the overall waiting and turnaround time. Round Robin provides fair CPU access to all processes, but its repeated time-slice execution results in higher waiting and turnaround times in this case.

5. Conclusion:

The experiment successfully compared **Round Robin (RR), Shortest Job First (SJF), and First Come First Serve (FCFS)** scheduling algorithms based on their Completion Time, Turnaround Time, and Waiting Time. From the results, **SJF performed the best** for the given processes, with the lowest Average TAT of **27.83** and Average WT of **16.83**. FCFS showed moderate performance with an Average TAT of **31.67** and Average WT of **20.67**, while RR had the highest Average TAT of **42.17** and Average WT of **31.17**. Although Round Robin provides fair CPU allocation and is suitable for time-sharing systems, it was less efficient for this particular workload. Therefore, **SJF is the most efficient choice for this dataset when the main goal is to minimize waiting and turnaround time**.

6. GitHub:

https://github.com/Rizon40/CSE-402-Operating-System-Lab